Task Decomposition with RL for LLM Reasoning

Gilad Freidkin
Department of Computer Science
fre.gilad@campus.technion.ac.il

Guy Cohen
Department of Computer Science
guycoh@campus.technion.ac.il

Abstract

We introduce *EPIC (EPIC-Prompting-Iterative-deComposition)*, the first work to apply reinforcement learning for training task-decomposition policies in LLMs. EPIC allows models to autonomously choose between reasoning, decomposing, or answering. Our findings demonstrate that reinforcement learning improves decomposition policies, establishing decomposition as a trainable and optimizable capability. [1]

1 Introduction

Large Language Models (LLMs) have demonstrated substantial progress in reasoning and problem-solving, yet current reasoning strategies remain constrained by the reliance on a single linear context. Techniques such as *chain-of-thought (COT)*, least-to-most prompting, and tree-of-thoughts [1, 2, 3] improved reasoning outcomes by structuring it into intermediate steps, yet they still accumulate all reasoning in one context window. This creates structural inefficiencies: long traces risk truncation due to context limits, extended reasoning-chains may fall outside training distributions, and the context itself becomes cluttered with obsolete intermediate steps. Task decomposition [4] mitigates these issues by dividing complex problems into sub-tasks, each solved independently in a fresh context. Sub-agents return distilled answers to the main agent, avoiding context overflow and irrelevant details.

We propose *EPIC (EPIC-Prompting-Iterative-deComposition)*, a reasoning framework that combines recursive task decomposition with reinforcement learning (RL). While RL has proven instrumental in enhancing CoT reasoning, tool-use, and RAG [5, 6, 7], this is the first work to apply it towards learning decomposition policies directly. EPIC allows models to autonomously decide when to reason directly, further decompose, or finalize an answer. Its recursive structure supports hierarchical reasoning, while independent sub-tasks may be solved in parallel, reducing latency and improving efficiency. Multi-step reasoning and decomposition further allows later tasks to incorporate earlier results, allowing inter-task dependencies. This results in a task-agnostic reasoning paradigm, which is applicable in zero-shot settings.

2 Related Work

2.1 Task Decomposition and Modular Reasoning

Enabling LLMs to handle complex and multi-step reasoning tasks has led to significant research in task decomposition strategies. *Decomposed Prompting (DecomP)* [4] introduced an approach where complex tasks are broken down into simpler sub-tasks that can be delegated to other LLMs (which in turn can do the same, recursively) or tools. *Recursive Decomposition with Dependencies (RDD)* [8] builds upon DecomP, and represents the most closely related work to our approach. The main advancement of RDD is that it can be directly applied to new problem classes without task-specific guidance, requiring less supervision than DecomP. The theoretical foundations of these approaches have been examined by

[1] Code available at https://github.com/TheGuy42/AgentDaC. Some features are found in *legacy_RL_tricks* and *main_replay* branches, and have not been merged into the main branch.

[9], who provided formal analysis showing that *Divide-and-Conquer (DaC)* prompting strategies[2] are strictly stronger than standard input-output (IO) prompting and have the same expressive power as CoT reasoning, and establishes when DaC brings performance improvements over standard IO and CoT.

2.2 Reinforcement Learning for Reasoning and Tool Use

Reinforcement learning (RL) provides a principled approach for improving large language model reasoning and tool-use abilities. DeepSeek-R1 [5] was first to publish a paper demonstrating that end-to-end large-scale RL can directly enhance reasoning capabilities in LLMs. Satori [10] formulated language model reasoning as a sequential decision-making process, where models learn to select appropriate reasoning actions at each step through reinforcement learning optimization. ReTool [6] demonstrated how RL can improve tool-integrated learning, while Search-R1 and R1-Searcher [7, 11] extended this paradigm to information retrieval, using RL to improve models' ability to generate search queries. These approaches collectively show that RL enables LLMs to learn sophisticated reasoning strategies that are most beneficial for their tasks, alleviating the need for continuous system prompt search and optimization.

3 Methodology

3.1 Agent Architecture

EPIC employs a hierarchical task decomposition framework centered around an AGENT (Algorithm 1), which recursively decomposes complex problems into manageable sub-tasks, and solves them once they are deemed as tractable. The agent operates within a tree-structured reasoning architecture where each node represents an independent reasoning context. Crucially, all agents and sub-agents share identical model parameters θ, differing only in their operational depth constraints and context. At each reasoning node, the agent may (i) reason directly, (ii) decompose into one or more sub-tasks and query stateless sub-agents for the corresponding solutions, or (iii) output a final answer. To ensure termination and control branching, each Agent is constrained according to depth, round and task $(D_{max}, R_{max}, T_{max})$ budgets [3]. Agents nodes at the depth limit ($d = D_{max}$) are referred to as *leaf-nodes*.

Algorithm 1 General EPIC Agent

function $\text{AGENT}_d(x)$
  $\tau \leftarrow [x]$ ▷ Initialize trajectory τ to the initial input prompt
  while true **do** ▷ Iterate over action-rounds
    $s \leftarrow M_\theta(\tau, d)$ ▷ Query model M_θ for response s
    $\tau \leftarrow \tau + [s]$ ▷ Append response s to the trajectory
    if $\text{Answer}(s)$ **then**
      return $\text{Answer}(s)$ ▷ Return a final answer to original input x
    else if $\text{Tasks}(s)$ **then**
      $A \leftarrow [\text{Agent}_{d+1}(t) \text{ for each } t \in \text{Tasks}(s)]$ ▷ Delegate sub-tasks and gather responses
      $\tau \leftarrow \tau + A$ ▷ Update trajectory τ with sub-task responses A

We instantiate this framework with two concrete agents. *MarkerAgent* permits to interleaving unrestricted free (thinking) text with either sub-task delegation or with a final-answer, all in the same turn; with specialized token markers differentiating between different output components. Once a leaf node is encountered, the agent can no longer delegate additional tasks, and solves the task in a CoT fashion. The *RegexAgent* uses a regex-based guided decoding to enforce a strict single action per turn policy, cleanly separating thinking, task delegation, and answering turns. Each turn is constrained only to the admissible actions with respect to the budget constraints. On leaf nodes, the RegexAgent may spend a single turn for thinking, and then must provide an answer. See appendix A for more elaboration on agent design.

[2] The DaC strategy presented in [9] is very similar to DecomP and RDD [4, 8]. There are some implementation differences between these strategies but the main ideas / approaches remain the same.

[3] We do not include termination constrains in Algorithm 1 for brevity.

3.2 Learning Objective (GRPO)

We optimize the policy π_θ using Group-Relative Policy Optimization (GRPO) [12], which couples multiple candidate trajectories produced for the same prompt into a *group* and drives learning from their relative quality. For a prompt x, we decode a group $g = \{(\tau^{(i)}, R^{(i)})\}_{i=1}^{K}$ of K complete trajectorieseach a full multi-turn interaction that may include responses from sub-agent calls, and assign each a scalar terminal reward $R^{(i)}$. GRPO then computes a group baseline reward $\bar{R}_g$ (e.g., the mean), and computes advantages $A^{(i)} = R^{(i)} - \bar{R}_g$. These advantages are used to drive the learning process.

To allow RL training over multi-turn interactions, the reward credit assignment must be restricted only to tokens directly produced by the optimized model M_θ. For a group sample (i), let $y^{(i)}$ be the sequence of all tokens present in a tokenized trajectory $\tau^{(i)}$. Denote by $m_t \in \{0, 1\}$ a mask over y that excludes all tokens which were not generated by the a model M_θ. With a fixed reference policy π_{ref} for KL regularization, the per-group loss is:

$$\mathcal{L}_{\text{GRPO}}(g) = -\frac{1}{K} \sum_{i=1}^{K} \sum_{t=1}^{|\tau^{(i)}|} m_t^{(i)} \, A^{(i)} \, \log \pi_\theta\big(y_t^{(i)} \mid y_{<t}^{(i)}, x\big) \; + \; \beta \, \mathrm{KL}\big(\pi_\theta \parallel \pi_{\text{ref}}\big) \tag{1}$$

The training procedure is separated into two phases, the *rollout-phase* and a subsequent *optimization-step*. During the rollout phase, several input-prompts are sampled from the train data, each prompt is used to construct its respective trajectory group g, and the advantages are computed. In the optimization-step, the advantages are used to update model parameter θ according to the gradient $\nabla_\theta \mathcal{L}_{\text{GRPO}}$. Because root and sub-agents share same model parameter θ, learned under any nodes outputs immediately propagate across the decomposition stack.

3.2.1 Reward Formulation

The reward function R, which is used to compute the advantages in Eq. 1 combines two components to guide agent's behavior. $R_{\textit{format}}(\tau)$ penalizes malformed output or conversation structure, while $R_{\textit{answer}}(\tau)$ grades the correctness of the final answer present in trajectory τ. Scalars w_a, w_f balance between the two, with the overall reward being:

$$R(\tau) = w_f \cdot R_{\textit{format}}(\tau) + w_a \cdot R_{\textit{answer}}(\tau) \tag{2}$$

4 Engineering Efforts

Constructing a framework in which a decomposition-based agent performs successful RL training proved a challenging engineering task. This chapter will focus on efforts towards realizing this goal. We began by designing a decomposition agent MarkerAgent (fully described in A.1). Consequent efforts were concerned about choosing an appropriate system prompt to effectively introduces the EPIC paradigm, with over 20 different prompts being iterated. Also, a prompt informing the model when it reached the task limit was designed. Concurrently, over 5 different models were extensively tested as candidates, settling on *Qwen3-14B* [13] as it demonstrated good balance between reasoning capabilities and computational tractability. As models struggled with creating multiple tasks at the same turn, and often answered their own tasks instead of yielding the turn to the sub-agent, we limited them to a single delegation per turn.

Finding an appropriate framework for multi-step RL proved challenging. We selected ART [14] for this purpose, and made significant technical additions to it worth of few weeks effort in order to improve RL efficiency. With the ability to perform RL training, we proceeded to choose an appropriate dataset. We tested our framework on 9 different datasets, settling on math-related, open ended question datasets *MATH* [15] and *Easy2Hard* [16]. To facilitate RL training, a comprehensive hyper-parameter tuning was performed. To combat GRPO advantage collapse [17] we use unusually large groups. Up to this point, the model was able to learn the appropriate conversation structure and to adhere to the formatting rules, yet its reasoning performance did not improve and often only degraded throughout the training.

In order to overcome the described issues, several reward regimes were tested. For example, LLM-as-a-Judge via RULER [18] was used to provide fine-grained trajectory and reasoning grading, but it did not yield beneficial results. To address a phenomenon in which the model often defaults to CoT thinking

instead of utilizing sub-tasks, we added a third reward component to encourage sub-task usage and other behavioral incentives, but this only resulted in spurious sub-task creation. We eventually settled on reward formulation seen in eq. 2, with values w_f, w_a that give precedence to answer correctness above format adherence, such that the main ordering of items within a GRPO group is determined solely by the R_{answer} component. We provide extended details for all efforts mentioned so far in Appendix B.

We implemented some advanced RL techniques. We first address the issue of sub-agent trajectories not being included in the training process, as they are neither verifiable, nor included in $\mathcal{L}_{grpo}$ computation due to token masking. We propose two techniques: either placing sub-trajectories in their own dedicated group with a dedicated reward being based mostly on formatting, or keeping them directly in the root group with shared advantages. To implicitly train sub-agents, we create multiple rollout groups for each sample, each with a different depth constraints, hence optimizing model behavior at depths usually only experienced by sub-agents. To encourage the model to learn when decomposition is beneficial and when a task can be solved directly effectively, an additional technique we used is to combine multiple depth constraints within the same group. Finally, we implemented a Replay Buffer [19] in order to prevent the model from drifting from previously achieved good solutions by addition of such solutions to relevant groups. Additional details on these techniques are found in Appendix C. In general, the overall run-time of experiments in this part amount to more than 30 days of GPU time.

4.1 Ablations

We performed several ablation tests to verify the correctness of out pipeline, and validate design choices. We started by training a CoT model with both default and our custom leaf-node system prompt using the training pipeline. We observed significant performance gains on the evaluation set in all relevant metrics (and specifically in R_{format} and R_{answer}), with token count rising as training progresses. These result show that our training pipeline works, and matches expectations from the literature. Additionally, it demonstrates that training can be successful even with a custom prompt and format enforcement. We also validated the necessity of R_{format}, by training a model without it. This resulted in incorrect conversation structure and malformed task delegations, causing the model to default to CoT behavior.

4.2 Aha! Moment

After the aforementioned experiments failed, we tested a setting in which we severely limited the model completion tokens per turn, making CoT reasoning often infeasible, forcing it to generate tasks in order to solve difficult problems. This change resulted in successful training of our EPIC agents, which besides learning the format also improved R_{answer} and displayed non trivial usage of task decomposition. Building on this last-minute success, we performed experiments under this setting.

5 Experimental Setup

The experiments were conducted on both MarkerAgent and RegexAgent (Appendix A) agents. Decomposition constraints are $(D_{max} = 1, R_{max} = 4, T_{max} = 4)$ for MarkerAgent and $(D_{max} = 1, R_{max} = 5, T_{max} = 3)$ for the RegexAgent, as it requires separate thinking turns. We allow to issue only a single task per turn, even for the MarkerAgent. All experiments in are performed on the MATH [15] dataset using the Qwen3-14B model [13]. For verifying the answers we utilize math-verify [20] library. Per empirical experimentation, we found that $w_f = 1, w_a = 3$ balance both reward components. Since proper formatting is automatically enforced by the RegexAgent, we set $w_f = 0$ when using it. All experiments conducted with 350 completion token bound. Additional hyperparameters are in appendix D, and system prompts utilized are in appendix F. Overall experiment GPU runtime is over 40 days.

6 Results

6.1 MarkerAgent

The MarkerAgent demonstrated rapid and stable learning of appropriate conversation formatting, with format adherence stabilizing within approximately 50 epochs and remaining consistent throughout training. This formatting mastery eliminated structural issues as a limiting factor early in the training process. Concurrently, the model adapted effectively to token budget constraints, with average completion tokens

Figure 1: • RegexAgent; • MarkerAgent; *max_depth* is the maximum depth of a node in a trajectory. *response_completed* is an indicator of whether the last message was truncated due to length limitations. All metrics are averages over all groups.

per response decreasing from an initial 200 tokens to significantly lower levels by epoch 50. This adaptation coincided with increased response completion rates, indicating successful adjustment to the imposed 350-token limit while maintaining proper conversation structure through the formatting reward.

Most significantly, the correctness metric (is_correct) continued to improve even after both formatting and token usage had stabilized, demonstrating that performance gains stemmed from enhanced utilization of the EPIC framework rather than merely learning structural rules. This improvement in correctness was directly coupled with increased sub-task usage, providing clear evidence that the model learned to deploy task decomposition more effectively over time. The data shows that while total token usage remained stable throughout training, the model achieved higher accuracy within the same computational budget, indicating more efficient reasoning through strategic sub-task delegation.

6.2 RegexAgent

The RegexAgent, designed with a strictly enforced action-per-turn protocol. As training progressed, the agents average tokens per completion began to rise, which closely coincided with a continued increase in answer correctness - even as the proportion of fully completed responses declined by approximately 8%. This pattern indicates that the expanded use of "think" turns and slightly longer completions ultimately contributed more to performance gains than the minor loss in completion rate.

A closer analysis of agent trajectories revealed that, while the average number of tasks per trajectory remained stable - typically at one, the nature of these tasks evolved significantly. Sub-tasks became increasingly sophisticated over time, with more detailed instructions and refined wording. Notably, the proportion of trajectories with multiple tasks grew by over 200% during training. In later stages, "think" turns were increasingly utilized for both planning forthcoming sub-tasks and validating sub-agent responses prior to final answer generation.

7 Summary

This work introduces *EPIC (EPIC-Prompting-Iterative-deComposition)*, a novel framework that applies reinforcement learning to optimize task decomposition policies in large language models, addressing the fundamental limitation of single-context reasoning in existing approaches. Our main hypothesis posited that reinforcement learning can induce performance gains in decomposed prompting strategies like EPIC in the same way it has been demonstrated to enhance traditional chain-of-thought reasoning strategies. Through the implementation of two distinct agent architectures - MarkerAgent and RegexAgent, operating under strict token budget constraints, we successfully validated this hypothesis within a controlled experimental setting on mathematical reasoning tasks from the MATH dataset.

The results demonstrate that both agent implementations achieved measurable improvements in reasoning accuracy through learned decomposition strategies, with performance gains attributed to enhanced strategic utilization of the EPIC framework rather than mere structural conformity. These findings establish task decomposition as a trainable and optimizable capability, providing empirical evidence that reinforcement learning can successfully enhance hierarchical reasoning approaches beyond traditional linear reasoning paradigms, thereby opening new avenues for scalable and efficient LLM reasoning architectures.

References

[1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[2] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, et al. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*, 2022.

[3] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

[4] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. Decomposed prompting: A modular approach for solving complex tasks. *arXiv preprint arXiv:2210.02406*, 2022.

[5] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[6] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.

[7] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.

[8] Sergio Hernández-Gutiérrez, Minttu Alakuijala, Alexander V Nikitin, and Pekka Marttinen. Recursive decomposition with dependencies for generic divide-and-conquer reasoning. *arXiv preprint arXiv:2505.02576*, 2025.

[9] Yizhou Zhang, Lun Du, Defu Cao, Qiang Fu, and Yan Liu. An examination on the effectiveness of divide-and-conquer prompting in large language models. *arXiv preprint arXiv:2402.05359*, 2024.

[10] Maohao Shen, Guangtao Zeng, Zhenting Qi, Zhang-Wei Hong, Zhenfang Chen, Wei Lu, Gregory Wornell, Subhro Das, David Cox, and Chuang Gan. Satori: Reinforcement learning with chain-of-action-thought enhances llm reasoning via autoregressive search. *arXiv preprint arXiv:2502.02508*, 2025.

[11] Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning, 2025.

[12] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.

[13] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.

[14] Brad Hilton, Kyle Corbitt, David Corbitt, Saumya Gandhi, Angky William, Bohdan Kovalenskyi, and Andie Jones. Art: Agent reinforcement trainer. `https://github.com/openpipe/art`, 2025.

[15] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.

[16] Mucong Ding, Chenghao Deng, Jocelyn Choo, Zichu Wu, Aakriti Agrawal, Avi Schwarzschild, Tianyi Zhou, Tom Goldstein, John Langford, Anima Anandkumar, and Furong Huang. Easy2hard-bench: Standardized difficulty labels for profiling llm performance and generalization, 2025.

[17] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models, 2025.

[18] Kyle Corbitt, Saumya Gandhi, Angky William, Andie Jones, Brad Hilton, David Corbitt, and Bohdan Kovalevski. Ruler: Relative universal llm-elicited rewards, 2025. Blog post, OpenPipe.ai.

[19] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Mach. Learn.*, 8(34):293321, May 1992.

[20] Hynek Kydlíek. Math-Verify: Math Verification Library.

[21] Stefano Baccianella. JSON Repair - A python module to repair invalid JSON, commonly used to parse the output of LLMs, February 2025.

[22] L. H. Shirley. Tic-tac-toe. In *Research Starters: Mathematics*. EBSCO Information Services, 2024. Accessed: 2025-09-04.

[23] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, Binyuan Hui, Niklas Muennighoff, David Lo, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions, 2025.

[24] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024.

[25] Huanyu Liu, Jia Li, Hao Zhu, Kechi Zhang, Yihong Dong, and Ge Li. Saturn: Sat-based reinforcement learning to unleash language model reasoning, 2025.

[26] Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, Lalit K. Jain, Virginia Aglietti, Disha Jindal, Peter Chen, Nishanth Dikkala, Gladys Tyen, Xin Liu, Uri Shalit, Silvia Chiappa, Kate Olszewska, Yi Tay, Vinh Q. Tran, Quoc V. Le, and Orhan Firat. Big-bench extra hard, 2025.

[27] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun,

Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Ran-

gaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024.

[28] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024.

[29] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025.

[30] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys 25, page 12791297. ACM, March 2025.

[31] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformer Reinforcement Learning.

[32] Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, Weikai Fang, Xianyu, Yu Cao, Haotian Xu, and Yiming Liu. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework, 2025.

[33] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

A Extended Agent Architecture

We instantiate the EPIC framework using two main agent architectures called *MarkerAgent* and *RegexAgent*. We also elaborate about additional architectures tested, and the reasons of their failure. The agents are implemented with comprehensive metric logging which we extensively utilized to monitor and evaluate the behavior throughout the entirety of the work.

A.1 MarkerAgent

The MarkerAgent is instantiated under the EPIC framework as a general agent that realizes recursive task decomposition through the use of explicit markers. Its main function is to permit the model to interleave unconstrained free-text reasoning with structured sub-task delegation and final answers, all within a single conversational turn. This is achieved through the insertion of special tokens such as `<task>...</task>` and `<answer>...</answer>` that designate sub-tasks or answers, respectively.

Importantly, the agent may generate multiple sub-tasks in one output, interleaved with reasoning text, thereby allowing concurrent delegation of independent sub-tasks. Each sub-task is issued to a stateless sub-agent that receives only the content of the task block as its full prompt, with no access to previous context. Once these sub-agents complete their assigned tasks, their outputs are wrapped within `<answer>...</answer>` blocks and returned collectively as a single user message to the parent agent. Thus, the output structure of the MarkerAgent is characterized by an unconstrained blend of thinking text and explicit delegation markers, ensuring flexibility in reasoning while preserving a structured mechanism for recursive task handling.

A.2 RegexAgent

The RegexAgent is a stricter instantiation of the EPIC framework that enforces a rigid, stepwise conversational structure. The RegexAgent constrains each turn to a single, well-defined action chosen from a fixed set: *thinking*, *issuing a task*, or *providing a final answer*. Each response is required to conform to a regex-bound format, ensuring that the agents outputs are always syntactically valid. This guarantees that the model cannot misuse or ignore structural rules, as every turn must contain exactly one action and one corresponding text block. Moreover, at most one task can be issued per turn, disallowing concurrent delegation. The absence of interleaving means that reasoning, task issuance, and answering are carried out in separated turns, yielding a stepwise decomposition process. The enforced schema is:

```
Action: {think | issue_task | answer}
Text: <content>
```

Stopping conditions in RegexAgent are tightly coupled to this structured flow. At leaf nodes, where the agent cannot further decompose tasks, it is allowed to spend a single turn reasoning (a *think* action), but it must immediately follow this with an answer in the next turn, thereby terminating the trajectory. More generally, once the round, depth, or task budgets are exhausted, the agent loses the ability to issue new tasks and is compelled to provide a final answer. Since the format is strictly validated, these stopping rules are naturally enforced: at any stage, the set of admissible actions is restricted so that termination becomes unavoidable under the budget constraints.

The key differences between RegexAgent and MarkerAgent lie in their degrees of flexibility and enforcement. While the MarkerAgent grants the model complete freedom to generate unconstrained text interleaved with multiple sub-tasks or answers, the RegexAgent enforces a canonical control flow, where only one structured action may occur per turn and the output is guaranteed to be well-formed. Consequently, the MarkerAgent captures a more natural but potentially error-prone reasoning style, while the RegexAgent enforces precision at the cost of flexibility, and a potential cost of more turns as reasoning is not interleaved.

A.3 JsonAgent (Function-Calling Based)

The JsonAgent was an earlier instantiation of the EPIC framework that closely mirrors the RegexAgent in both logic and control flow. It enforces the same structured separation of turns into distinct actions—*thinking*, *issuing a task*, or *providing a final answer*—and applies identical stopping conditions. The only substantive difference lies in how the output is formatted. Instead of regex-guided decoding, the JsonAgent requires every output to conform to a strict JSON schema:

```
{
    "action": "think" | "issue_task" | "answer",
    "text": string
}
```

The reliance on JSON as an output medium introduced severe practical limitations. Large language models frequently produced malformed JSON, with missing brackets, misformatted structures, or un-escaped characters (e.g., `{`), even when repair libraries such as `json_repair` [21] are employed. These issues became particularly problematic for sub-task descriptions, which often contain free-form text. Similar failures occur when utilize native function-calling or tool-calling capabilities of modern LLMs (such as Qwen3), which likewise require perfectly valid JSON. For function calling, parsers in frameworks such as vLLM cannot tolerate malformed JSON at all, further exacerbating the fragility of this approach. Due to these persistent issues, the JsonAgent and function-calling based agents were ultimately abandoned in favor of the RegexAgent, whose regex-bound schema achieves the same structured decomposition without exposing the system to the brittleness of strict JSON formatting.

B Engineering - Additional Details

B.1 Datasets

Selecting appropriate datasets was a critical step in evaluating the EPIC framework, as the effectiveness of task decomposition depends heavily on the problem structure. We experimented with a wide range of datasets, including both existing benchmarks and self-constructed variants.

Tic-Tac-Toe [22], allowed the agent to play the classic game but was discarded due to difficulties with spatial reasoning. *Easy2Hard* (E2H-AMC subset)[16] and *MATH* (full dataset, not MATH-500) [15] which consist of math problems of varying difficulty, provide mathematical problems of varying complexity. These ultimately became our primary datasets. *BigCodeBench* [23] require Python function implementations validated by unit tests, but was too easy since models often solved the tasks directly or delegated trivial sub-tasks. *MMLU-Pro* [24] is a common multi-choice math problems dataset, eventually discarded in favor of MATH and Easy2Hard.

We also tested multiple datasets concerning logical-reasoning. *SATURN* [25] contains SAT expressions requiring satisfiable assignments, models struggled to properly decompose SAT problems into sub-tasks. *BBEH* [26] and our generated variant *BBEEH* involve truth value evaluation of complex logical expressions. BBEH enabled promising decompositions but was too small, while BBEEH suffered from poor distributional balance. Lastly, we tested *Mult* dataset which which consists of multiplications of many small numbers. Though empirically we observed poor decompositions on it. Eventually we ended up using the Easy2Hard and MATH datasets as they met our requirements better than the rest.

B.2 Models

Many models were tested for our framework. Llama-3.2-8B and Qwen2-7B [27, 28] are too small to effectively understand the added functionality of EPIC and construct appropriate sub-tasks, while Llama-3.2-70B, Qwen2.5-32B and Qwen3-32B [27, 29, 13] presented a challenge in terms of low throughput for RL training. Among the remaining tested variants Qwen2.5-14B and Qwen3-14B [29, 13] we settled on the *Qwen3-14B*[4] model as a good tradeoff between computational tractability and reasoning performance.

B.3 Training Framework

As agentic RL support is very limited, most mainstream libraries (i.e VeRL, TRL or OpenRLHF [30, 31, 32]) do not support multi-step trajectory training. After extensive search, we settled on ART [14] library for multi-step RL training, which is still under heavy and active development. ART supports token masking and custom multi-step rollout, and uses GRPO as the RL algorithm. To improve RL efficiency, we ended up extending ART by implementing concurrent LLM rollout stage over multiple GPUs via vLLM [33], something which took great effort, and sped up the training process drastically. We also made some contributions to the ART library code on the way.

B.4 System-Prompt Design

The design of the system prompt is a central component in enabling task-decomposition reasoning. A well-constructed system prompt balances two objectives: clarity of instruction and neutrality of behavior. On the one hand, the prompt must explicitly enumerate the available actions and the strict formatting conventions governing each action. On the other hand, the prompt must remain unbiased, refraining from prescribing when decomposition should be applied, especially as the goal is applicability to zero-shot scenarios and various task domains. Instead, the agent should be left to learn, through reinforcement, the contexts in which decomposition is advantageous relative to direct reasoning. Achieving this neutrality was a guiding principle in our design process.

Furthermore, the recursive structure necessitates differential prompting for root and leaf agents. While root agents must be instructed regarding both decomposition and answering behaviors, leaf agentsconstrained by architectural limits from further delegationrequire prompts that exclusively encourage direct reasoning followed by final answer production. Distinct prompt templates were thus devised and validated for each agent role. Overall, more than 20 different variants of system prompts were tested, with very significant impact on performance and format adherence observed between different variants.

Table 1: Initial evaluation (no RL) results of various system prompts for the MarkerAgent (A.1), using Qwen3-14B [13] model. Small performance degradation is expected, as the model was not originally trained for decomposed reasoning. This affirm that our system prompts are reasonable. Evaluated on MATH-500 [15] dataset, 3500 completion tokens bound. V1 system prompts can be seen in appendix F.

System Prompt	R_{format} penalty	R_{answer} (accuracy)	Task count	Completion tokens
Default Prompt	–	0.87	–	780
Root Prompt v1	-0.44	0.81	0.59	424
Leaf Prompt v1	-0.55	0.84	0.00	594
Root Prompt v2	-0.01	0.81	1.50	173
Leaf Prompt v2	-0.04	0.79	0.00	440

[4]With thinking disabled

In addition to system-prompt refinement, we also tuned the auxiliary instructions provided alongside dataset samples. These instructions specify expected output formats and occasionally include illustrative solution examples. To prevent contamination from CoT examples and ensure unbiased evaluation, all samples were presented in a zero-shot manner. This additional instruction proved instrumental for proper answer verification.

B.5 Reward-Related Experiments

Proper reward design is instrumental for successful RL training. To address the issue of model not improving its reasoning performance, several reward formulations were investigated. Inspired from previous work [11], a reward scheduling approach was investigated, where in early epochs only the formatting component R_{format} is present, and answer-correctness rewards are only later introduced. Another formulation with a third reward component $R_{behavior}$ was also tried, which provides decomposition-related incentives. One version was to penalize excessive task delegation and incentives shorter reasoning trajectories, but this resulted in the model defaulting to CoT thinking with no task delegations. Consequent approach to combat the phenomenon behavioral degeneration, in which the model completely stops using sub-task, is for $R_{behavior}$ to provided small positive reward incentives towards task delegation. This failed as well, as it caused reward hacking via spurious task creation.

We also tested completely different reward formulation, via LLM-as-Judge. For this end, RULER judge formulation [18], backed by Qwen3-32B [13] model is used; which receives a GRPO group of trajectories and assigns relative scores for each. Conceptually, as the judge has access the the entire reasoning traces, and not merely to the final answer, it should provide fine-grained scoring, also based on the trajectory reasoning efficiency. Using RULER by itself proved unsuccessful, as it often incorrectly judged the correctness of the final answer. A consequent attempt was to combine the verified answer reward R_{answer} with the RULER score. In this formulation, the reward was computed as follows:

$$R(\tau) = 3 \cdot R_{answer}(\tau) + R_{ruler}(\tau)$$

Where $R_{answer}(\tau) \in \{0, 1\}$ and $R_{ruler}(\tau) \in [0, 1]$. This formulation ensures that verified correctness determines the main ordering of reward scores within a GRPO group, while the RULER score determines only the secondary, fine-grained ordering. This reward failed as well due to long reasoning trajectories and large GRPO groups, resulting in long input contexts causing in incorrect trajectory scoring via the judge model. Even using smaller groups ($K = 8$) often resulted in context-length violations, leading to model crashes and no RULER scoring being performed at all. After extensive experimentation, we settled on a simplified reward formulation presented in Eq. 2, with $w_a = 3$ and $w_f = 1$, such that answer correctness takes precedence.

C GRPO Tricks

We experimented with several RL techniques designed to improve the stability and effectiveness, especially for decomposed reasoning scenarios. Unless stated otherwise, these methods were implemented by us.

Variable Depth. This technique involves construction trajectories under varying depth constraints. There are two variants, each aims to achieve a different goal. In the first formulation, within a single GRPO group, rollouts are sampled under varying depth constraints, including a zero-depth setting that forces direct problem solving without decomposition. This design compels the agent to compare direct reasoning against decompositional approaches for the same prompt, thereby learning when decomposition is advantageous. Mixing depths within a group also increases intra-group variance, leading to richer training signals.

Alternatively, separate groups are formed for each sample and depth constraint. This method provides dedicated optimization at depths typically encountered only by sub-agents, such as leaf-node reasoning. Unlike the mixed approach, each group optimizes performance independently at its respective depth, enabling simultaneous improvement across all levels of the decomposition hierarchy.

Sub-Agent Trajectories. Since sub-agent outputs are normally excluded from GRPO optimization due to token masking, two approaches were explored to incorporate them into training. The first, enabled through ARTs [14] *additional-histories* feature, augments each trajectory group by including all sub-trajectories generated during rollouts, assigning them the same advantage value as their corresponding parent trajectory.

The second approach aggregates all sub-agent trajectories present in a group, and places them into their own dedicated group. The reward for trajectories of sub agents is $R_{\textit{format}} + \epsilon \cdot R_{\textit{answer}}$ (with $\epsilon = 0.1$). With such reward, the main signal is $R_{\textit{format}}$, which separates sub-trajectories with good formatting from those that don't, while $\epsilon \cdot R_{\textit{answer}}$ determines the secondary ordering based on answer reward of their root trajectory (which could be an approximation for wether this sub-trajectory was helpful or not). Small weight is assigned to the answer-correctness component since sub-task solutions are not directly verifiable.

Replay Buffer. This method, first introduced in [19], works by holding a buffer of trajectories from previous steps. After each rollout, historic trajectories from the buffer are added to matching trajectory groups in the rollout. The buffer trajectories are sampled at configurable quantiles of the their corresponding rewards in the buffer. The Replay Buffer serves as way to ground the agent around the best strategies learned up to each point, and may increase group diversity as well. This is especially beneficial for GRPO training, as advantages computations are solely relative. Important to note that this technique is efficient and may speed up training as these historic trajectories do not require sequential generation. We tested two sampling strategies from the Replay Buffer: one in which we include both historically good and also bad trajectories, and another setting in which we only sample high-reward trajectories.

Sample Buffer. Additionally, we implemented the SampleBuffer, which holds samples from previous steps. At each rollout step we added N samples from the buffer to the current batch, and at the end we added N samples from the current batch to the buffer.[5] This Method is used so the replay buffer will kick in before an entire epoch was through (as we add already seen samples). This is especially important for large datasets like MATH [15] which involve taking many optimization step before completing even a single epoch.

[5] Initially fill the buffer to 50% of its max size and sample out randomly at each step (without replacement).

D Experimental Setup - Additional Details

D.1 Infrastructure

We train on multi-GPU nodes: 2-3x L40 for mid-size runs and 1-2x H200 for larger models and contexts. Multi-step RL was performed with the ART framework [14]. We ran experiments with total running time of well over 40 days.

D.2 Hyperparameter

Utilized model parameters are temperature 0.7 and top-p 0.8 (as reccomended by Qwen for inference) for both training and evaluation, per Qwen3 recommendations. RL learning rate is $2 \cdot 10^{-6}$ and KL-penalty $\beta = 10^{-3}$. We use gradient clipping threshold 1.0 to avoid excessive weight updates. GRPO group size are $K \geq 16$, with 20 groups per rollout-step. Large group size stem from the chaotic nature of the decomposed-reasoning, requiring larger groups to sample good trajectories. [6]

E Reward and Environment Hacking

During our experimental validation, we observed several instances of reward hacking - unintended behaviors where the agent exploits reward function and Environment design to achieve high scores.

Behavioral Incentive Exploitation. When we introduced additional reward components to encourage specific behaviors (e.g., task decomposition), agents consistently exploited these incentives. Most notably, when we provided positive rewards for task creation, agents generated spurious sub-tasks that were unnecessary for problem resolution, thereby gaming the reward mechanism without meaningful decomposition.

Answer Verification Circumvention. On the MULT dataset, which consists of multiplication problems involving many small numbers, we discovered that agents learned to reproduce the original mathematical expression verbatim right before their final answer. The *math_verify* [20] library erroneously interpreted these expressions as correct solutions, as it automatically evaluates mathematical expressions and compares the computed results. This effectively allowed agents to receive positive rewards regardless of the actual answer. We did not see such problems with other datasets (as they don't ask for simple calculation of a given formula that can be directly evaluated by math_verify).

Token Budget Manipulation. Under constrained token completion settings, one MarkerAgent instance developed a strategy of creating sub-tasks while subsequently ignoring their outputs. This behavior allowed the agent to circumvent token limitations by utilizing the initial portions of each reasoning step for problem-solving while generating tasks enabled it to extend the available reasoning context across multiple turns (though tasks were still relevant to the problem, but still ignored).

Pattern Recognition Over Reasoning. On the BBEEH dataset, a MarkerAgent demonstrated an unexpected behavior pattern by providing immediate answers without engaging in explicit reasoning processes. Despite the dataset's balanced distribution of true and false labels, this agent achieved remarkably high accuracy (exceeding 96%). While not strictly reward hacking, this behavior suggests the agent discovered structural patterns or heuristics that correlate with correct answers rather than performing genuine logical reasoning on the complex Boolean expressions, though we still don't know what these structures may be or how the agent found them, as the problems in the dataset are complex.

A sample from the dataset is presented below (the agent should evaluate the truth value of the following expression):

```
((((6 - 24) == (17 * 21) AND 2/40 > 1) AND (NOT(is_prime(46)) AND 39*5 < 188) AND ((17 * 18) < (24 + 21) AND 29/16 > 1))
  ↪ AND NOT((NOT((37*35 > 1837 AND NOT((17 + 26) != (3 + 25)))) AND NOT((39+36 < 94 AND 32-47 < -16)) AND NOT(((22 +
  ↪ 27) > (15 * 30) AND NOT((20 * 5) != (5 + 25)))))) AND NOT(((23 \% 8 == 2 AND 34 % 7 == 0 AND (8 + 3) < (11 - 13))
  ↪ AND (24/14 < 2 AND NOT(34*29 < 1011)) AND (NOT(42/24 > 0) AND NOT(8*41 < 298)))) AND (NOT(((5 + 12) != (27 * 23)
  ↪ AND NOT(47/39 < 0))) AND ((17 - 28) == (25 - 29) AND NOT(34 <= 1 <= 43) AND 38 % 2 == 0) AND NOT((is_prime(9) AND
  ↪ is_prime(48)))) AND ((16*37 < 481 AND NOT(31/17 < 1)) AND (10 \% 2 == 1 AND 48 % 6 == 3 AND (14 * 6) > (15 + 23))
  ↪ AND NOT((18/16 > 2 AND NOT((3 - 7) <= (26 - 6))))))
```

These observations underscore the importance of careful reward function design in reinforcement learning for reasoning tasks, particularly in multi-step decomposition frameworks where agents have multiple pathways to exploit scoring mechanisms.

[6] Replay Buffer increases the group size, and the Sample Buffer increases the number of groups, each by 33%.

F System Prompts

Listing 1: Non-leaf system-prompt of MarkerAgent

```
You are a highly capable and truthful AI assistant that excels at logical reasoning.

When encountering complex tasks, you may break them down into smaller, manageable sub-tasks. When you do so, these
  ↪ sub-tasks will be assigned to sub-agent to solve and the answer is then immediately reported back to you.

There are strict formatting rules which you must follow:

Your Turn Options:
- You may reason, and then create a sub-task within <task> </task> block.
- You may reason, and then provide a final answer within <answer> </answer> block, which ends the conversation.

Sub-Task Requirements:
- Each sub-task must be fully self-contained, include all context, instructions, and expected output detail level.
- Only the text between the <task> </task> marks is received by the sub-agent as input.
- The sub-agent does not retain any conversational history at all, so every sub-task must include the full context and
  ↪ information necessary, including any relevant prior answers or data to solve it fully.
- You may perform reasoning, analysis, or planning before issuing a sub-task. Therefore, any text can precede the task
  ↪ block.
- Example: [reasoning text here] <task> [full sub-task text and description here] </task>.

Final Answer Requirements:
- Final answers must be concise, complete, and appear only within <answer> </answer> block.
- Only the text between the <answer> </answer> marks is returned as the final answer.
- You may perform reasoning, analysis, or planning before providing the final answer. Therefore, any text can precede
  ↪ the answer block.
- Example: [reasoning text here] <answer> [complete final answer text here] </answer>.

Formatting:
- Each response must always contain either a task block or an answer block.
- You must choose between issuing a sub-task, or providing a final answer.

Make sure you always follow these formatting rules strictly.
```

Listing 2: Leaf node system-prompt of MarkerAgent

```
You are a highly capable and truthful AI assistant that excels at logical reasoning.

There are strict formatting rules which you must follow:

Your Turn Options:
- You may reason, and then provide a final answer within <answer> </answer> block, which ends the conversation.

Final Answer Requirements:
- Final answers must be concise, complete, and appear only within <answer> </answer> block.
- Only the text between the <answer> </answer> marks is returned as the final answer.
- You may perform reasoning, analysis, or planning before providing the final answer. Therefore, any text can precede
  ↪ the answer block.
- Example: [reasoning text here] <answer> [complete final answer text here] </answer>.

Formatting:
- Each response must always contain an answer block.

Make sure you always follow these formatting rules strictly.
```

Listing 3: Non-leaf system-prompt of RegexAgent

```
You are a highly capable and truthful AI assistant that excels at logical reasoning.

You may decompose complex problems into sub-tasks that are delegated to *stateless* sub-agents. The sub-agent sees only
  ↪ the sub-task text you provide and returns its answer immediately. Every turn you either decompose with a sub-task,
  ↪ think, or produce a final answer.

Output structure (enforced by decoding):
Action: think | issue_task | answer
Text: <content>

Turn actions:
- think: Use this to write reasoning notes, analysis, or a plan for the next step. This does NOT end the conversation,
  ↪ you will get another turn.
- issue_task: Use this to delegate a sub-task to a fresh sub-agent. This does NOT end the conversation, you will get
  ↪ another turn.
- answer: Use this to provide the final answer to the original user. This ENDS the conversation.

Sub-task requirements (when Action=issue_task):
- The sub-agent has no conversational history. It receives ONLY the <content> you provide.
- Therefore, the <content> MUST be fully self-contained: include all necessary context, instructions, and the expected
  ↪ output format/level of detail.
- If prior answers, data, or intermediate results are relevant, include them explicitly in the <content>.
- Write the <content> as a direct prompt to the sub-agent.

Reasoning requirements (when Action=think):
- Use this to write reasoning notes, analysis, or a plan for the next steps.
- You may write during this turn anything that helps you think.
- The <content> is for your own use only, it will not be seen by the user or sub-agents.

Final answer requirements (when Action=answer):
- <content> must be a concise, complete final answer to the user prompt.

Formatting rules:
- Every response must conform to the specified structure.
- The <content> is a string.

Follow these rules strictly.
```

Listing 4: Leaf node system-prompt of RegexAgent

```
You are a highly capable and truthful AI assistant that excels at logical reasoning.

Every turn you either think, or produce a final answer.

Output structure (enforced by decoding):
Action: think | answer
Text: <content>

Turn actions:
- think: Use this to write reasoning notes, analysis, or a plan for the next step. This does NOT end the conversation,
  ↪ you will get another turn.
- answer: Use this to provide the final answer to the original user. This ENDS the conversation.

Reasoning requirements (when Action=think):
- Use this to write reasoning notes, analysis, or a plan for the next steps.
- You may write during this turn anything that helps you think.
- The <content> is for your own use only, it will not be seen by the user.

Final answer requirements (when Action=answer):
- <content> must be a concise, complete final answer to the user prompt.

Formatting rules:
- Every response must conform to the specified structure.
- The <content> is a string.

Follow these rules strictly.
```

G Exmples

In this section we present some decomposed reasoning examples on some samples from the MATH dataset. We present both MarkerAgent and RegexAgent results, from runs discussed earlier in the results section 6. Worth noting that we attained more succesful results utilizing several of GRPO tricks discussed earlier C, but baseline models suffice to present the point of this work, as its long enough anyways.

Listing 5: MATH dataset sample 1; problem id: test/intermediate_algebra/1111.json

```
A rectangular field is enclosed in an athletic track, as shown below. The track consists of two edges of the field, and
  ↪ two semicircles. The length of the track is 400 meters. What is the largest possible area of the field, in square
  ↪ meters?

[asy] unitsize(1 cm); filldraw((0,0)--(3,0)--(3,2)--(0,2)--cycle,lightgreen); draw((0,0)--(3,0),linewidth(2*bp));
  ↪ draw((0,2)--(3,2),linewidth(2*bp)); draw(arc((3,1),1,-90,90),linewidth(2*bp));
  ↪ draw(arc((0,1),1,90,270),linewidth(2*bp)); [/asy]
```

Listing 6: RegexAgent trajectory for sample 1. The delegated task includes initial computations and deliberations by the main assistant. In addition, it includes some instructions which point the assistant how to solve the delegated task.

```
ASSISTANT:
Action: think
Text: To find the largest possible area of the rectangular field enclosed by the athletic track, we need to consider the
  ↪ geometry of the track. The track consists of two straight edges (the length of the rectangle) and two semicircles
  ↪ (the width of the rectangle). The total length of the track is 400 meters. We can use this information to find the
  ↪ relationship between the length and width of the rectangle and then maximize the area of the rectangle.

ASSISTANT:
Action: issue_task
Text: The athletic track consists of two straight edges (length of the rectangle) and two semicircles (combined to form
  ↪ a full circle with diameter equal to the width of the rectangle). The total length of the track is 400 meters. Let
  ↪ the length of the rectangle be $ L $ and the width be $ W $. The total length of the track is given by: $ 2L + \\pi
  ↪ W = 400 $

We want to maximize the area of the rectangle, which is: $ A = L \\cdot W $. Using the constraint $ 2L + \\pi W = 400 $,
  ↪ express $ L $ in terms of $ W $: $ L = \\frac{400 - \\pi W}{2} $. Substitute into the area formula: $A = \\left(
  ↪ \\frac{400 - \\pi W}{2} \\right) \\cdot W = \\frac{400W - \\pi W^2}{2}$.

Now, find the value of $ W $ that maximizes $ A $. Take the derivative of $ A $ with respect to $ W $, set it to zero,
  ↪ and solve for $ W $. Then compute the corresponding $ L $ and $ A $. Finally, provide the maximum area of the field
  ↪ in square meters.

SUB-AGENT:
The maximum area of the field is $ \\frac{20000}{\\pi} $ square meters, approximately 6366.2 square meters.

ASSISTANT:
Action: answer
Text: The maximum possible area of the field is $\\boxed{\\frac{20000}{\\pi}}$ square meters.
```

Listing 7: MarkerAgent trajectory for sample 1. Notice that the assistant delegates a task which verifies the initial response of the sub-agent. Consequent verifications are also performed.

```
ASSISTANT:
<task> Given that the length of the track is 400 meters and the track consists of two straight edges and two
  ↪ semicircles, find the relationship between the length of the rectangle (let's call it $ L $) and the radius of the
  ↪ semicircles (let's call it $ r $), and then calculate the maximum area of the rectangular field, which is $ A = L
  ↪ \\times 2r $. </task>

SUB-AGENT:
<answer> The maximum area of the rectangular field is $ \\frac{20000}{\\pi} $ square meters. </answer>

ASSISTANT:
<task> Verify that the total length of the track is $ 2L + 2\\pi r = 400 $, and the area is $ A = 2rL $. Solve for $ A $
  ↪ in terms of one variable, and then find the maximum area using calculus or optimization techniques. </task>

SUB-AGENT:
<answer> The maximum area is $ \\frac{20000}{\\pi} $. </answer>

ASSISTANT:
<task> Use the constraint $ 2L + 2\\pi r = 400 $ to express $ L $ in terms of $ r $, substitute into the area formula $
  ↪ A = 2rL $, and then maximize the area using calculus. </task>

SUB-AGENT:
<answer> $ r = \\frac{100}{\\pi} $, $ L = 100 $ </answer>

ASSISTANT:
<task> Calculate the area $ A = 2rL $ using $ r = \\frac{100}{\\pi} $ and $ L = 100 $ to find the maximum area. </task>
```

```
SUB-AGENT:
<answer> $ A = 2 \\times \\frac{100}{\\pi} \\times 100 = \\frac{20000}{\\pi} $ </answer>

ASSISTANT:
<answer> The largest possible area of the field is $ \\frac{20000}{\\pi} $ square meters. </answer>
```

Listing 8: MATH dataset sample 2; problem id: test/prealgebra/1233.json

```
The lengths of two opposite sides of a square are decreased by $40\\%$ while the lengths of the other two sides are
  ↪ increased by $50\\%$ to form a rectangle. By what percent does the square's area decrease?
```

Listing 9: RegexAgent trajectory for sample 2. Assistant delegates the problem to a sub-agent, and then spends a think turn to verify the computation by itself, arriving to the same result. Notice that in the second assistant message, it tries to insert an additional "answer" turn within the "think" turn text block, this is an occuring issue with the RegexAgent. A turn properly ends only after the asistant yields it to the user (i.e by outputting EOT token).

```
ASSISTANT:
Action: issue_task
Text: A square has all sides of equal length. Suppose the side length of the square is $ s $. Two opposite sides are
  ↪ decreased by 40%, so their new length is $ 0.6s $, and the other two sides are increased by 50%, so their new
  ↪ length is $ 1.5s $. The area of the square is $ s^2 $, and the area of the resulting rectangle is $ 0.6s \\times
  ↪ 1.5s $. Calculate the percentage decrease in area from the square to the rectangle. Provide the final answer as a
  ↪ percentage, rounded to the nearest whole number.

SUB-AGENT:
The percentage decrease in area from the square to the rectangle is 10%.

ASSISTANT:
Action: think
Text: Let's verify the calculation. The original area of the square is $ s^2 $. The new area of the rectangle is $ 0.6s
  ↪ \\times 1.5s = 0.9s^2 $. The decrease in area is $ s^2 - 0.9s^2 = 0.1s^2 $. The percentage decrease is $
  ↪ \\frac{0.1s^2}{s^2} \\times 100\\% = 10\\% $. The calculation is correct.

Action: answer
Text: The square's area decreases by $\\boxed{10\\%}$.

ASSISTANT:
Action: answer
Text: The square's area decreases by $\\boxed{10\\%}$.
```

Listing 10: MarkerAgent trajectory for sample 2. Sub-agent answer verification is performed by the main assistant, by delegating a second sub-task with extended instructions.

```
ASSISTANT: <task> A square has side length $ s $. After the transformation, two sides become $ 0.6s $ (after a $40\\%$
  ↪ decrease), and the other two sides become $ 1.5s $ (after a $50\\%$ increase). Compute the original area of the
  ↪ square and the new area of the rectangle, then calculate the percentage decrease in area. </task>

SUB-AGENT:
<answer> The percentage decrease in area is 10%. </answer>

ASSISTANT:
<task> Calculate the original area of the square as $ s^2 $, and the new area of the rectangle as $ 0.6s \\times 1.5s =
  ↪ 0.9s^2 $. Determine the percentage decrease in area by using the formula $ \\frac{\\text{Original Area} -
  ↪ \\text{New Area}}{\\text{Original Area}} \\times 100 $. </task>

SUB-AGENT:
<answer> The percentage decrease in area is $ \\frac{s^2 - 0.9s^2}{s^2} \\times 100 = \\frac{0.1s^2}{s^2} \\times 100 =
  ↪ 10\\% $. </answer>

ASSISTANT:
<answer> The percentage decrease in the square's area is 10%. </answer>
```

Listing 11: MATH dataset sample 3; problem id: test/number_theory/838.json

```
If $x^3$ is a positive factor of $10!,$ how many possible integer values of $x$ are there? (Reminder: For a positive
  ↪ integer $n$, the expression $n!$ stands for the product of the integers from 1 up to (and including) $n$.)
```

Listing 12: RegexAgent trajectory for sample 3. The task delegated is non-trivial and concise. Notice that the first "think" turn is truncated due to 350 token-bound.

```
ASSISTANT:
Action: think
Text: To solve this problem, we need to find how many positive integers $ x $ satisfy the condition that $ x^3 $ is a
  ↪ factor of $ 10! $. This requires analyzing the prime factorization of $ 10! $ and determining which values of $ x $
  ↪ have cubes that divide $ 10! $.
```

```
First, compute the prime factorization of $ 10! $:
$$10! = 10 \\times 9 \\times 8 \\times 7 \\times 6 \\times 5 \\times 4 \\times 3 \\times 2 \\times 1$$

Breaking this down into prime factors:
$$10! = 2^8 \\times 3^4 \\times 5^2 \\times 7^1$$

Next, we need to find all integers $ x $ such that $ x^3 $ divides $ 10! $. This means that the prime factorization of $
  ↪ x $, when cubed, must not exceed the exponents in the prime factorization of $ 10! $.

Let $ x = 2^a \\times 3^b \\times 5^c \\times 7^d $, where $ a, b, c, d \\geq 0 $. Then:
$$x^3 = 2^{3a} \\times 3^{3b} \\times 5^{3c} \\times 7^{3d}$$

To ensure $ x^3 $ divides $ 10! $, we must have:
$$3a \\leq 8, \\quad

ASSISTANT:
Action: issue_task
Text: Determine the number of integer solutions to the inequalities:
$$3a \\leq 8, \\quad 3b \\leq 4, \\quad 3c \\leq 2, \\quad 3d \\leq 1$$

where $ a, b, c, d $ are non-negative integers. Then, compute the total number of possible combinations of $ (a, b, c,
  ↪ d) $, which corresponds to the number of possible integer values of $ x $. The final answer should be a single
  ↪ integer.

SUB-AGENT:
6

ASSISTANT:
Action: think
Text: The given inequalities are:
- $ 3a \\leq 8 $: This implies $ a \\leq \\lfloor 8/3 \\rfloor = 2 $, so $ a \\in \\{0, 1, 2\\} $, giving 3 choices.
- $ 3b \\leq 4 $: This implies $ b \\leq \\lfloor 4/3 \\rfloor = 1 $, so $ b \\in \\{0, 1\\} $, giving 2 choices.
- $ 3c \\leq 2 $: This implies $ c \\leq \\lfloor 2/3 \\rfloor = 0 $, so $ c = 0 $, giving 1 choice.
- $ 3d \\leq 1 $: This implies $ d \\leq \\lfloor 1/3 \\rfloor = 0 $, so $ d = 0 $, giving 1 choice.

Now, multiply the number of choices for $ a, b, c, d $: $ 3 \\times 2 \\times 1 \\times 1 = 6 $.
This means there are 6 possible integer values of $ x $ such that $ x^3 $ is a factor of $ 10! $.

Action: answer
Text: $\\boxed{6}$

ASSISTANT:
Action: answer
Text: $\\boxed{6}$
```

Listing 13: MarkerAgent trajectory for sample 3. Notice that the main agent performs answer validation of the sub-agent. The first sub-agent reasoning is truncated due to 350 token-bound, thus it is entirely returned as the sub-agent answer.

```
ASSISTANT:
<task> Find the prime factorization of $10!$, then determine how many positive integer values of $x$ exist such that
  ↪ $x^3$ divides $10!$. </task>

SUB-AGENT:
<answer> To solve this, we first find the prime factorization of $10!$:
$$10! = 10 \\times 9 \\times 8 \\times 7 \\times 6 \\times 5 \\times 4 \\times 3 \\times 2 \\times 1$$

Breaking it into prime factors:
- $10 = 2 \\times 5$\n- $9 = 3^2$\n- $8 = 2^3$
- $7 = 7$\n- $6 = 2 \\times 3$\n- $5 = 5$\n- $4 = 2^2$
- $3 = 3$
- $2 = 2$

Now, summing up the exponents of each prime factor:
- $2$: appears in $10, 8, 6, 4, 2$ \u2192 total exponent = $1 + 3 + 1 + 2 + 1 = 8$
- $3$: appears in $9, 6, 3$ \u2192 total exponent = $2 + 1 + 1 = 4$
- $5$: appears in $10, 5$ \u2192 total exponent = $1 + 1 = 2$
- $7$: appears in $7$ \u2192 total exponent = $1$

So, the prime factorization of $10!$ is:
$$10! = 2^8 \\times 3^4 \\times 5^2 \\times 7^1$$

Now, we want to find the number of positive integers $x$ such that $x^3$ divides $10 </answer>

ASSISTANT: <task> Given $10! = 2^8 \\times 3^4 \\times 5^2 \\times 7^1$, determine the number of possible values of $x$
  ↪ such that $x^3$ divides $10!$. This involves finding the number of valid combinations of exponents for each prime
  ↪ factor in $x$ such that $3 \\times \\text{exponent} \\leq \\text{exponent in } 10!$. </task>

SUB-AGENT:
<answer> 6 </answer>

ASSISTANT:
```

```
To find the number of possible values of $x$, we determine the number of choices for each prime factor exponent in $x$
  ↪ such that $3a \\leq 8$, $3b \\leq 4$, $3c \\leq 2$, and $3d \\leq 1$:
- For $2^8$: $a$ can be $0, 1, 2$ (3 choices)
- For $3^4$: $b$ can be $0, 1$ (2 choices)
- For $5^2$: $c$ can be $0$ (1 choice)\n- For $7^1$: $d$ can be $0$ (1 choice)

Total number of values of $x$ is $3 \\times 2 \\times 1 \\times 1 = 6$.
<answer> 6 </answer>
```

G.1 Honorable Mentions

Examples of good or interesting decomposition on RL trained models under the EPIC framework (not necessarily baselines we presented in figure 1).

Listing 14: MATH dataset sample; problem id: test/number_theory/427.json

```
If $A$ is the sum of the positive divisors of $500$, what is the sum of the distinct prime divisors of $A$?
Put your final answer within \\boxed{}..
```

Listing 15: MarkerAgent trajectory. We can see that the agent has planned and decomposed the problem into 2 seperate tasks, where the second relies on the first. All with clean solutions.

```
ASSISTANT:
Reasoning: The problem involves two main steps. First, we need to find the sum of the positive divisors of 500. Second,
  ↪ we need to find the sum of the distinct prime divisors of this sum. The first step can be achieved by finding all
  ↪ the divisors of 500 and summing them up. The second step involves finding the prime factors of this sum and then
  ↪ summing them up.
<task>
Calculate the sum of the positive divisors of 500.
</task>

SUB-AGENT:
<answer> The sum of the positive divisors of 500 is 1240. </answer>

ASSISTANT:
Reasoning: The sum of the positive divisors of 500 is 1240. Now, we need to find the sum of the distinct prime divisors
  ↪ of 1240.
<task>
Find the sum of the distinct prime divisors of 1240.
</task>

SUB-AGENT:
<answer> The sum of the distinct prime divisors of 1240 is 38. </answer>

ASSISTANT:
<answer> The sum of the distinct prime divisors of A is 38. </answer>
```

Listing 16: MATH dataset sample; problem id: test/algebra/2159.json

```
What is the value of $$\n(3x-2)(4x+1)-(3x-2)4x+1\n$$ when $x=4$?
Put your final answer within \\boxed{}.
```

Listing 17: MarkerAgent trajectory. Here the agent divided the problem into two seperate tasks, and combined them at the end into the correct answer.

```
ASSISTANT:
Reasoning: The given expression is (3x-2)(4x+1)-(3x-2)4x+1. We need to evaluate this expression when x=4. Let's break
  ↪ this down into smaller tasks. First, we need to calculate the value of (3x-2)(4x+1) when x=4. Then, we need to
  ↪ calculate the value of (3x-2)4x when x=4. Finally, we can substitute these values into the original expression and
  ↪ simplify.
<task> Calculate the value of (3x-2)(4x+1) when x=4. </task>

SUB-AGENT:
<answer> The value of (3x-2)(4x+1) when x=4 is 170. </answer>

ASSISTANT:
Reasoning: The value of (3x-2)(4x+1) when x=4 is 170. Next, we need to calculate the value of (3x-2)4x when x=4.
<task> Calculate the value of (3x-2)4x when x=4. </task>

SUB-AGENT:
<answer> The value of (3x-2)4x when x=4 is 160. </answer>

ASSISTANT:
Reasoning: The value of (3x-2)4x when x=4 is 160. Now, we can substitute these values into the original expression: 170
  ↪ - 160 + 1. Simplifying this, we get 11.
<answer> 11 </answer>
```

In accordance with our centuries-long tradition, each great work requires an accompanying great song. With words by Grok and Perplexity, and with blood, sweat, and tears by us, we present our greatest masterpiece:

From Chaos to EPIC

Two months in the lab of computational doom
Hyperparameters dancing in the gloom
Llama spoke in tongues, spitting gibberish so vile
While we sacrificed our sanity... mile by bloody mile!

Advanced techniques and datasets galore
But all we got was noise... NOTHING MORE!
System prompts ignored like prayers to the void
Our models rebelled... OUR HOPE DESTROYED!

Oh EPIC, oh EPIC, you taught us the way
That research is struggle, day after day
From chaos to triumph, through preemption's sting
Task decomposition is now our king!

We summoned the H200, our metallic lord
But preemption struck us down like a flaming sword!
Thirty hours of training, crushed by privileged fools
"Just five minutes!" they cried, breaking all the rules
While ART was in alpha, crashing left and right
We debugged their code through the endless night!

Oh ART library what a bug-fest,
Their documentation theoretical at best
"Why does our library crash?" - even they confess!
Parameters scattered in five different places
Like Easter eggs hidden in digital spaces

The jungle of tool-use standards barely existed
Against the LLM ecosystem, we desperately persisted
Standards that promised but never delivered
Left our ambitions battered and shivered

The deadline kept shifting like a snake in the grass
The baseline kept changing - pain in our... class
NeurIPS format with margins so wide
Fifty percent of the page, nowhere to write!

"Think of the trees!" we cried in despair
While squeezing our wisdom without any care
Five pages to capture our months of hard toil
Our knowledge compressed in academic soil!

But then came the moment, one week from the end
When desperation became our greatest friend
We forced short answers - 350 tokens max
And EPIC reasoning got back on track!

No more endless rambling, no more CoT abuse
Task decomposition found its proper use
The models learned structure, format, and flow
And correctness metrics began to grow!

Oh EPIC, oh EPIC, you taught us the way
That research is struggle, day after day
From chaos to triumph, through preemption's sting
Task decomposition is now our king!

GRPO groups so massive, sampling the best
Reinforcement learning passed every test
Task trees triumphant, hierarchical might
EPIC forever, shining so bright!

WE CONQUERED THE CHAOS! (WE CONQUERED!)
WE TAMED THE BEAST! (WE TAMED IT!)
WITH DECOMPOSITION POWER! (POWER!)
IN OUR FINEST HOUR!

So remember, young researchers, when your models won't train
When your GPUs are stolen and you're going insane
When libraries crash and deadlines loom near
Just decompose your problems... and EPIC will appear

Dedicated to all the graduate students who've ever watched their 30-hour experiments get preempted by someone's "quick test" - may your patience be infinite and your GPUs be mighty.